

GROUP 12: SENTIMENT ANALYSIS SYSTEM

Social Media Sentiment Analysis Using NLP

University Sentiment Intelligence System **(UniSentiment)**

Project Report

Group Number(s):	12
Index Number(s):	0151, 0052, 0195, 0220, 0199, 0145
Course / Programme:	Intro to Intelligent Systems (INF 402)
Lecturer:	Dr. Patrick Gyaase
Submission :	Within the Week

Table of Contents

Table of Contents	1
1. Introduction	2
2. Problem Statement.....	3
3. Objectives	4
4. Methodology.....	4
4.1 Data Collection.....	4
4.2 NLP Pipeline Design	5
4.3 Tools and Technologies	5
4.4 Testing Approach	6
5. System Design.....	6
5.1 Architecture Overview	6
5.2 Data Flow	6
5.3 Aspect Keyword Dictionary.....	6
5.4 User Interface Design.....	7
6. Implementation	7
6.1 Sentiment Scoring Pipeline.....	7
6.2 Aspect-Specific Scoring	8
6.3 Insight and Alert Generation	8
6.4 Web Application Layer.....	8
7. Testing and Results	9
7.1 Overall Results	9
7.2 Aspect-Level Results	9
7.3 Alert Generation Results.....	10
7.4 Explainability Examples.....	10
7.5 Known Limitation Observed During Testing.....	10
8. Conclusion.....	10
9. Recommendations	11
10. References	12

1. Introduction

Universities generate a continuous stream of feedback from students through social media posts, course evaluation forms, and informal online reviews. This feedback covers nearly every aspect of campus life, including teaching quality, dining facilities, internet connectivity, parking, library services, administrative support, campus security, and general infrastructure. While this volume of feedback is valuable, it is also difficult to process manually. A single semester can generate hundreds or thousands of comments, and reading through them one by one to identify recurring problems is time-consuming, inconsistent, and prone to human bias.

This project, titled University Sentiment Intelligence System (UniSentiment), addresses this challenge by applying Natural Language Processing (NLP) techniques to automatically classify the sentiment of student comments and to identify which specific aspect of university life each comment refers to. Rather than producing a single overall sentiment score for an entire batch of comments, the system performs aspect-based sentiment analysis, meaning it can detect that a single comment such as "the professors are great but the Wi-Fi is terrible" expresses two different sentiments about two different aspects within the same sentence.

The system was built using Python, the Natural Language Toolkit (NLTK), and TextBlob, and is delivered as an interactive web application built with Flask. The application allows a user to paste or type student comments directly into a browser-based dashboard, run the analysis with a single click, and immediately see the results visualised through charts, tables, and a searchable comment explorer. This report documents the design, implementation, testing, and evaluation of the system.

2. Problem Statement

University administrators and student affairs offices typically rely on manual reading of feedback, periodic surveys, or simple keyword counting to gauge student satisfaction. These approaches share several weaknesses that this project sets out to solve:

- Manual review does not scale. As the number of comments grows, the time required to read and categorise them grows linearly, making it impractical to process feedback in real time.
- Basic sentiment tools only classify a whole comment as positive, negative, or neutral, without identifying which specific aspect of the university the sentiment relates to. A comment that is overall negative may still contain positive remarks about one department and negative remarks about another.
- Simple keyword-matching approaches fail to account for negation ("not good" is not the same as "good") and sarcasm ("oh great, another broken elevator" is not genuinely positive despite containing the word "great"), both of which are common in informal student writing.
- Without a way to track sentiment trends over time, it is difficult for management to notice when a problem is emerging before it becomes severe, or to confirm whether an intervention actually improved student sentiment.

This project therefore aims to build a tool that automatically classifies sentiment per aspect, accounts for negation and sarcasm, and presents the results in a way that is immediately actionable for university management, without requiring any manual reading of the underlying comments.

3. Objectives

The objectives of this project are as follows:

1. To design and implement an aspect-based sentiment analysis engine capable of detecting at least eight distinct aspects of university life within a single comment: professors, cafeteria, Wi-Fi, parking, library, administration, campus safety, and facilities.
2. To classify the sentiment of each comment, and of each aspect mentioned within a comment, as Positive, Negative, or Neutral using a combination of TextBlob's polarity scoring and a custom negation-handling layer.
3. To detect and correctly handle negation (e.g. "not good", "never works") so that the sentiment score is flipped or adjusted appropriately rather than being misclassified by surface-level keyword matching.
4. To detect likely sarcasm using pattern-based heuristics and to flag such comments separately so that their sentiment is not misinterpreted at face value.
5. To provide explainable output for every classification, showing the user exactly which words and which NLP steps led to the final sentiment label, rather than presenting an unexplained black-box score.
6. To generate actionable insights automatically, identifying which aspects most urgently need improvement ("Fix First"), which aspects are performing well and should be maintained ("Keep Doing"), and which aspects show a divided or controversial opinion among students.
7. To present all of the above through an interactive, visual, browser-based dashboard rather than a plain console printout, so that the results are usable by non-technical university staff.
8. To package the entire system as a single, easily deployable Python application that runs entirely offline with no dependency on external paid APIs.

4. Methodology

The system was developed following an iterative, modular approach. Development was split into two layers: an NLP processing engine responsible for all language analysis, and a presentation layer responsible for collecting input and displaying results. This separation allows the NLP logic to be tested and verified independently of the web interface.

4.1 Data Collection

Because this is a prototype intended to demonstrate the sentiment analysis pipeline rather than to perform a live social media scrape, a curated dataset of sixty (60) realistic student comments was hand-written to closely resemble genuine student feedback. The dataset was deliberately constructed to include a wide variety of NLP challenges, including straightforward positive and negative statements,

mixed-sentiment comments mentioning two or more aspects with opposing sentiment, comments containing explicit negation, and comments containing sarcasm. This deliberate variety ensures that the system's negation-handling and sarcasm-detection logic can be properly exercised and demonstrated. The system also accepts freely typed or pasted comments from the user at runtime, so it is not limited to the bundled sample dataset.

4.2 NLP Pipeline Design

Each comment passes through the following sequence of processing steps:

1. **Tokenisation:** The comment is split into individual lowercase word tokens using NLTK's `word_tokenize` function.
2. **Stopword Removal:** Common English stopwords (e.g. "the", "is", "a") are filtered out using NLTK's built-in stopwords corpus, leaving only content-bearing words for keyword analysis.
3. **Aspect Detection:** The comment (and, for aspect-specific scoring, the individual sentences within it via NLTK's `sent_tokenize`) is checked against a keyword dictionary covering eight university aspects, using regular-expression word-boundary matching to avoid partial-word false matches.
4. **Base Sentiment Scoring:** TextBlob is used to compute a polarity score from -1.0 (very negative) to +1.0 (very positive), and a subjectivity score from 0.0 (factual) to 1.0 (opinionated), for the comment or for the aspect-relevant sentences.
5. **Negation Adjustment:** A custom sliding-window algorithm scans the token list for negation words (e.g. "not", "never", "can't") and, if one is found within three tokens of a sentiment-bearing word, flips or reduces the polarity score accordingly.
6. **Sarcasm Detection:** A set of regular-expression patterns checks for common sarcastic constructions (e.g. "oh sure", "very cool", excessive punctuation, trailing ellipses). If matched, the polarity is forced negative regardless of the surface wording.
7. **Label Assignment:** The final adjusted polarity score is mapped to a Positive, Negative, or Neutral label using fixed thresholds.
8. **Evidence Extraction:** TextBlob's part-of-speech tagger is used to extract the specific adjectives that contributed positively or negatively to the score, which are surfaced to the user as explainability evidence.

4.3 Tools and Technologies

The following tools and libraries were used in the implementation of this project:

- Python 3 — the core programming language for the entire system.
- NLTK (Natural Language Toolkit) — used for word tokenisation, sentence tokenisation, and stopwords filtering.
- TextBlob — used for sentiment polarity scoring, subjectivity scoring, and part-of-speech tagging.
- Flask — a lightweight Python web framework used to serve the application as a browser-based dashboard and to expose REST API endpoints.
- Chart.js — a JavaScript charting library, loaded via CDN, used to render the doughnut chart and bar chart in the browser.

- Regular Expressions (re module) — used throughout for keyword matching, aspect detection, and sarcasm pattern matching.

4.4 Testing Approach

Testing was carried out at two levels. At the unit level, the core scoring functions (`score_sentiment`, `detect_aspects`, `detect_negation`, `detect_sarcasm`) were exercised against the 60-comment sample dataset and their output was manually verified against expected classifications. At the system level, the Flask application's routes were tested end-to-end, sending sample comments to the `/analyse` and `/single` endpoints and confirming the returned JSON matched the expected structure and values. Full details and results are provided in Section 7, Testing and Results.

5. System Design

The system follows a two-tier architecture consisting of a backend NLP engine and a Flask-based web layer that serves both the user interface and a small REST API. This section describes the architecture, the data flow through the system, and the structure of the aspect keyword dictionary that underpins aspect detection.

5.1 Architecture Overview

The backend is split into two Python modules. `sentiment_engine.py` contains all NLP logic and has no dependency on Flask, meaning it can be imported and tested independently or reused in a different interface (for example, a command-line tool) without modification. `main.py` contains the Flask application, including all HTTP routes and the embedded HTML/CSS/JavaScript that make up the single-page dashboard. This separation of concerns follows standard software engineering practice and made the system considerably easier to test and debug during development.

5.2 Data Flow

When a user submits comments through the dashboard, the following sequence occurs:

1. The browser sends a POST request containing the list of comments (one per line from the textarea) to the `/analyse` endpoint as JSON.
2. Flask receives the request and passes each comment through the `sentiment_engine` functions: `detect_aspects()`, `score_sentiment()`, and `score_aspect_sentiment()` for every aspect found.
3. The results for every comment are aggregated into per-aspect statistics (percentage positive/negative/neutral, average compound score, mention count), and into overall statistics across the whole batch.
4. Insight-generation logic ranks the aspects by their average score to produce the Fix First, Keep Doing, and Controversial lists, and an alert-generation function flags any aspect whose negative percentage crosses a defined threshold.
5. The complete results object is serialised to JSON and returned to the browser, where JavaScript renders it into the dashboard's metric cards, charts, tables, and comment list without requiring a page reload.

5.3 Aspect Keyword Dictionary

Aspect detection is implemented as a dictionary that maps each of the eight aspects to a list of associated keywords and phrases. A comment is considered to mention an aspect if any of its keywords appears in the comment as a whole-word match. The table below summarises the aspects and a sample of their associated keywords.

Aspect	Sample Keywords
Professors	professor, lecturer, faculty, teacher, instructor, TA, office hours, syllabus, advisor
Cafeteria	cafeteria, food, meal, dining, lunch, breakfast, dinner, menu, vegan, prices
Wi-Fi	wi-fi, wifi, internet, network, connection, bandwidth, signal, streaming
Parking	parking, lot, car park, vehicle, ticket, permit, shuttle, potholes
Library	library, librarian, book, study room, journal, database, research, thesis
Administration	administration, admin, office, paperwork, portal, appeal, scholarship, registration
Campus Safety	safety, security, safe, unsafe, guard, patrol, escort, lighting, emergency
Facilities	facilities, building, classroom, gym, lab, equipment, AC, heating, elevator

Table 1: Summary of the eight aspects and a sample of their associated keywords.

5.4 User Interface Design

The dashboard is organised into four functional zones. An input zone at the top of the page allows the user to either type a single comment for instant analysis or paste multiple comments into a textarea for batch analysis. A metrics zone displays four summary cards showing the total number of comments analysed and the percentage breakdown of positive, negative, and neutral sentiment. A visualisation zone presents a doughnut chart of the overall sentiment split alongside a grouped bar chart comparing positive and negative percentages across all eight aspects. Finally, an exploration zone provides a filterable, searchable list of every individual comment, where clicking a comment expands an explainability panel showing the exact TextBlob score, the negation/sarcasm flags, the positive and negative keywords detected, and the per-aspect breakdown for that specific comment.

6. Implementation

This section describes the implementation of the core NLP functions in more technical detail, including the specific algorithms used for negation handling, sarcasm detection, and aspect-specific scoring.

6.1 Sentiment Scoring Pipeline

The function `score_sentiment()` forms the core of the system. Given a piece of text, it first tokenises the text using NLTK and obtains a raw polarity score from TextBlob. It then calls a negation-detection function which performs a sliding-window search: for every negation word found in the token list (such as "not", "never", or "can't"), the function inspects the following three tokens for a content word. If a content word

is found within that window, the comment is treated as containing a negated sentiment, and the polarity score is inverted. In the specific case where the original polarity was negative (as in "not bad"), the score is inverted and halved in magnitude rather than fully flipped, since "not bad" is understood to be mildly positive rather than strongly positive.

After the negation adjustment, the text is checked against a list of sarcasm regular-expression patterns, including common ironic interjections such as "oh sure" and "very cool", as well as structural cues such as multiple consecutive exclamation marks or a trailing ellipsis. If any pattern matches, the polarity score is forcibly set to a strongly negative value, since sarcastic comments in the dataset are, by construction, expressions of frustration disguised as praise.

The final polarity score is clamped to the valid range of -1.0 to +1.0 and mapped to a label using fixed thresholds: scores at or above +0.05 are labelled Positive, scores at or below -0.05 are labelled Negative, and scores in between are labelled Neutral. To support the explainability requirement, the function also uses TextBlob's part-of-speech tagger to extract up to five adjectives that contributed positively and up to five that contributed negatively to the final score.

6.2 Aspect-Specific Scoring

A naive approach to aspect-based sentiment analysis would apply the comment's overall sentiment score to every aspect mentioned within it. This would be inaccurate for mixed-sentiment comments such as "the professors are great but the Wi-Fi is terrible", where the overall score would average out the strongly positive and strongly negative halves into a misleadingly neutral result for both aspects. To avoid this, the function `score_aspect_sentiment()` first uses NLTK's sentence tokeniser to split the comment into individual sentences, then filters to only those sentences containing a keyword for the target aspect, and finally runs the full sentiment scoring pipeline on just that filtered text. This means that in the example above, the system correctly scores "professors" as strongly positive based only on the first half of the sentence, and "Wi-Fi" as strongly negative based only on the second half.

6.3 Insight and Alert Generation

Once every comment in a batch has been scored, the system aggregates the per-aspect compound scores and ranks the aspects from most negative to most positive average score. The three aspects with the lowest average score, provided their negative percentage is 40% or higher, are surfaced as "Fix First" recommendations. The three aspects with the highest average score, provided their positive percentage is 60% or higher, are surfaced as "Keep Doing" recommendations. Aspects whose positive and negative percentages both fall between 35% and 65% are flagged as "Controversial", indicating a genuinely divided body of student opinion that may warrant further investigation rather than a simple fix. Separately, an alert-generation function checks every aspect's negative percentage against two thresholds, raising a HIGH severity alert at 60% or above and a MEDIUM severity alert at 40% or above, so that the most urgent problems are visually flagged in the dashboard's Alerts panel.

6.4 Web Application Layer

The Flask application exposes five routes. The root route ("/") serves the dashboard's HTML, CSS, and JavaScript as a single embedded string, which avoids the need for a separate templates folder and keeps the entire frontend in one file for ease of distribution. The "/load_sample" route returns the 60 built-in sample comments as JSON, allowing the dashboard's "Load Sample" button to populate the textarea instantly. The "/single" route accepts one comment and returns its full analysis for the Quick Analyse feature. The "/analyse" route accepts a batch of comments and returns the complete aggregated results object described in Section 5.2. The "/export_csv" route re-runs the analysis on the submitted comments and streams the results back as a downloadable CSV file, allowing the data to be opened in spreadsheet software for further analysis or for inclusion in this report's appendices.

7. Testing and Results

The system was tested by running the full analysis pipeline against the bundled 60-comment dataset and verifying both the aggregate statistics and a sample of individual comment classifications. This section presents the actual results produced by the system at the time of writing.

7.1 Overall Results

Across all 60 comments in the sample dataset, the system classified 57% as Positive, 28% as Negative, and 15% as Neutral. This aligns with the dataset's intended composition, which leans positive overall while still containing a meaningful number of negative and mixed-sentiment comments to exercise the negation and sarcasm handling logic.

7.2 Aspect-Level Results

The table below shows the actual aspect-level breakdown produced by the system, including the percentage of mentions classified as each sentiment, the average compound score, and the total number of times each aspect was mentioned across the dataset.

Aspect	Positive	Negative	Neutral	Avg Score	Mentions
Cafeteria	62%	25%	12%	-0.023	8
Library	57%	0%	43%	+0.376	7
Campus Safety	57%	14%	29%	+0.269	7
Facilities	56%	25%	19%	+0.219	16
Professors	50%	7%	43%	+0.220	14
Administration	50%	0%	50%	+0.186	8
Wi-Fi	33%	25%	42%	+0.030	12
Parking	30%	40%	30%	-0.053	10

Table 2: Actual aspect-level sentiment breakdown produced by the system on the 60-comment sample dataset.

These results correctly identify Parking as the weakest-performing aspect, with the highest negative percentage (40%) and the only negative average compound score alongside Cafeteria. This matches the underlying dataset, in which parking-related comments were deliberately weighted toward complaints about fees, potholes, and ticketing.

7.3 Alert Generation Results

Running the alert-detection logic against the aspect summary and the simulated 14-day sentiment trend produced five alerts: one MEDIUM severity alert for Parking (40% negative mentions), and four HIGH severity alerts triggered by single-day drops in overall positive sentiment of 20 percentage points or more within the simulated trend window. This confirms that both the aspect-level threshold check and the day-over-day trend check are functioning as designed.

7.4 Explainability Examples

The following examples, taken directly from system output, demonstrate the negation and sarcasm handling in practice.

Example 1 — Sarcasm correctly overriding a literally positive word:

Comment: “Oh wow, another parking ticket. Thanks university, very cool. Maybe build more parking.”

- TextBlob's raw polarity score (based purely on word choice) was +0.314, driven by the word “cool”.
- The sarcasm detector matched the pattern for “oh wow” and “very cool”, forcing the final compound score to -0.514 and the label to Negative.
- This demonstrates that the sarcasm layer successfully prevents a sarcastic compliment from being misread as genuine praise, which a plain TextBlob-only system would have failed to catch.

Example 2 — Straightforward positive classification:

Comment: “The professors are absolutely amazing and always willing to help after class.”

- Raw and final polarity scores matched at +0.425, correctly labelled Positive, with “amazing” and “willing” extracted as the supporting positive keywords.

7.5 Known Limitation Observed During Testing

Testing also surfaced a genuine limitation worth documenting honestly rather than omitting. The comment “I expected to hate it here but professors have completely changed my perspective” was classified as Negative (compound score -0.267), despite being intended as an overall positive statement. This occurs because TextBlob's lexicon-based scoring weighs the word “hate” heavily negative and does not recognise the contrastive structure introduced by “but”, which should discount the sentiment of the first clause in favour of the second. This is a known weakness of lexicon-based sentiment tools in general, and is documented here as a direction for future improvement (see Section 9, Recommendations) rather than presented as a solved problem.

8. Conclusion

This project set out to build a sentiment analysis system capable of going beyond simple positive/negative classification to deliver aspect-based insight into student feedback about university life. The resulting system, UniSentiment, successfully detects eight distinct aspects of university life within free-text comments, scores sentiment per aspect rather than per comment, and layers custom negation-handling and sarcasm-detection logic on top of TextBlob's base polarity scoring to address two of the most common failure modes in simple sentiment classifiers.

Testing against a 60-comment dataset confirmed that the system correctly identifies the weakest-performing aspect (Parking, at 40% negative sentiment) and the strongest-performing aspects (Library and Campus Safety, both above +0.25 average compound score), and that it generates appropriate severity-ranked alerts when negative sentiment crosses defined thresholds. The sarcasm-handling example documented in Section 7.4 demonstrates that the system can correctly override a surface-level positive word when it is used ironically, which a naive keyword-counting approach would fail to do.

The project also surfaced a genuine limitation in lexicon-based sentiment scoring when handling contrastive sentence structures (Section 7.5), and this limitation is documented honestly rather than concealed, both because it is methodologically important and because it directly motivates the recommendations for future work outlined in the following section.

Overall, the project demonstrates a complete, working pipeline from raw student comment text to an interactive, explainable, browser-based intelligence dashboard, fulfilling all eight objectives stated in Section 3 of this report.

9. Recommendations

Based on the testing results and the limitations observed during development, the following improvements are recommended for future iterations of this system:

1. Replace or augment the lexicon-based TextBlob scorer with a transformer-based sentiment model (such as a fine-tuned BERT or DistilBERT classifier) to better handle contrastive sentence structures of the type identified in Section 7.5, where a clause beginning with “but” should be weighted more heavily than the clause preceding it.
2. Expand the aspect keyword dictionary using a data-driven approach, such as TF-IDF keyword extraction over a larger real-world dataset of student comments, rather than the manually curated keyword lists currently used, to improve recall on aspects expressed using vocabulary not anticipated by the development team.
3. Integrate a live data source, such as a university feedback form API or a social media listening tool, so that the system can analyse real, continuously arriving comments rather than only the bundled sample dataset or manually pasted text.
4. Add persistent storage (for example, a lightweight database such as SQLite) so that analysis results and historical trends can be saved across sessions, rather than being recalculated each time the application restarts.

5. Extend the sarcasm-detection module beyond regular-expression pattern matching toward a trained sarcasm-classification model, since regular expressions can only catch sarcasm that follows a small number of known surface patterns and will miss more subtle or novel forms of irony.
6. Add user authentication and role-based access so that the dashboard could be safely deployed for use by multiple university staff members with different permission levels (for example, read-only access for general staff versus full export access for administrators).

10. References

Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media. (NLTK documentation: <https://www.nltk.org/>)

Loria, S. (2018). TextBlob: Simplified Text Processing. TextBlob documentation: <https://textblob.readthedocs.io/>

Pang, B., & Lee, L. (2008). Opinion Mining and Sentiment Analysis. Foundations and Trends in Information Retrieval, 2(1-2), 1-135.

Liu, B. (2012). Sentiment Analysis and Opinion Mining. Morgan & Claypool Publishers.

Flask Documentation. Pallets Projects. <https://flask.palletsprojects.com/>

Chart.js Documentation. <https://www.chartjs.org/docs/>

Python Software Foundation. Python 3 Language Reference. <https://docs.python.org/3/>